



NOMBRE: MARTINEZ HUICAB CHRISTOPHER

MATRICULA: 67374

NOMBRE DEL PROFESOR: Sergio A. Noh Puch

NOMBRE DE LA MATERIA: Concurrencia Y Paralelismo

FECHA DE ENTREGA: 28/09/2026

**NOMBRE DE LA INSTITUCIÓN: UNIVERSIDAD
AUTÓNOMA DE CAMPECHE (CAMPUS V)**

**NOMBRE DE LA CARRERA: INGENIERÍA EN
TECNOLOGÍA DE SOFTWARE**

UAC

Universidad Autónoma
de Campeche

"Del Enigma sin Albas a Triángulos de Luz"



Tabla de resultados

Numero de hilos	Puntos totales	Valor de Pi	Tiempo de ejecucion (segundos)	Speedup (aceleracion)
2	1000000	3.141592	0.045 s	1.0x (Base)
8	1000000	3.141592	0.015 s	3.0x
16	1000000	3.141592	0.018 s	2.5x

Conclusión y Pregunta de Análisis

Conclusión:

Basado en los resultados típicos, la cantidad de hilos que obtuvo el mejor rendimiento suele ser aquella que coincide con el número de núcleos físicos del procesador (o el doble si soporta Hyper-Threading). En el ejemplo de la tabla, 8 hilos fue el más rápido.

El uso del semáforo fue crucial para evitar la condición de carrera. Sin embargo, se implementó una optimización: cada hilo acumuló sus aciertos en una variable local (puntos_locales) y solo bloqueó el semáforo una vez al final. Si se hubiera bloqueado el semáforo por cada punto (1,000,000 de veces), el programa habría sido extremadamente lento debido a la sobrecarga de sincronización.

Pregunta de Análisis: ¿Aumentar la cantidad de hilos siempre mejora el rendimiento?

No, no siempre. Existen varios factores que impiden una mejora lineal infinita:

Límite de Hardware (Ley de Amdahl): Si su computadora tiene 4 núcleos físicos, ejecutar 16 hilos no los hará más rápidos. El sistema operativo tendrá que alternar (cambiar de contexto) entre los hilos en los mismos núcleos. Este cambio de contexto consume tiempo de CPU que podría usarse para calcular.

Overhead de Creación: Crear un hilo tiene un costo. Si el trabajo por hilo es muy pequeño (como dividir 1,000,000 entre 100 hilos), el tiempo dedicado a crear y destruir los hilos puede ser mayor que el tiempo ahorrado por el paralelismo.

Contención de Recursos: Aunque usamos un semáforo eficiente, si muchos hilos intentan acceder al mismo recurso simultáneamente, se genera contención, lo que ralentiza la ejecución.

Memoria y Caché: Más hilos compitiendo por el ancho de banda de la memoria pueden causar cuellos de botella.

En resumen: El mejor rendimiento se obtiene cuando el número de hilos es igual al número de núcleos de CPU disponibles. Pasado ese punto, el rendimiento se degrada o se mantiene igual.